

UNIVERSIDAD TECNOLÓGICA NACIONAL
Tecnatura Universitaria en Programación a Distancia

TRABAJO PRÁCTICO INTEGRADOR
SISTEMA DE GESTIÓN DE DATOS DE PAÍSES EN PYTHON

Materia: Programación I

Docente: Cátedra de Programación I

Integrantes:

- Navas, Adalberto
- Tejeda, Santiago

Fecha de Entrega: Junio 2026

ÍNDICE

1. INTRODUCCIÓN Y DESCRIPCIÓN DEL PROYECTO	Pág. 3
2. MARCO TEÓRICO	Pág. 4
2.1. Condicionales (Estructuras de Control de Flujo)	Pág. 4
2.2. Funciones	Pág. 4
2.3. Listas	Pág. 4
2.4. Diccionarios	Pág. 4
2.5. Ordenamientos (Algoritmo Bubble Sort)	Pág. 5
2.6. Estadísticas Básicas en Programación	Pág. 5
2.7. Manejo de Excepciones (Try-Except)	Pág. 5
2.8. Persistencia de Datos (Archivos CSV)	Pág. 5
3. DECISIONES TÉCNICAS Y ARQUITECTURA	Pág. 7
3.1. Estructura de Datos Seleccionada	Pág. 7
3.2. Diagrama de Flujo Principal	Pág. 7
3.3. Evidencia de Ejecución (Capturas de Pantalla)	Pág. 7
4. DIFICULTADES Y CONCLUSIONES	Pág. 8
5. BIBLIOGRAFÍA / WEBGRAFÍA	Pág. 10
6. ENLACES DEL PROYECTO (REPOSITORIO Y VIDEO)	Pág. 11

INTRODUCCIÓN Y DESCRIPCIÓN DEL PROYECTO

El presente proyecto consiste en el diseño y desarrollo de un sistema interactivo de consola para la gestión de información sobre países, desarrollado íntegramente en Python 3. La aplicación fue concebida como el Trabajo Práctico Integrador para la materia Programación I de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional.

El sistema permite centralizar la manipulación de datos demográficos y geográficos de diferentes naciones. Entre sus principales prestaciones se encuentran la carga dinámica de registros desde un archivo de texto estructurado (CSV), la inserción de nuevos elementos, la actualización de parámetros esenciales como población y superficie, la realización de búsquedas con coincidencias parciales, el filtrado avanzado por rangos y continentes, el ordenamiento de registros a través de algoritmos de ordenación interna y la generación de reportes estadísticos consolidados.

MARCO TEÓRICO

Condicionales (Estructuras de Control de Flujo)

Son estructuras que permiten desviar el flujo de ejecución lineal de un programa basándose en el resultado (verdadero o falso) de una expresión lógica o booleana. En Python se implementan mediante las palabras clave `if`, `elif` y `else`.

Allen B. Downey explica que la potencia de las computadoras radica en su capacidad de tomar decisiones lógicas. Define a las estructuras condicionales como "ejecuciones condicionales" que permiten comprobar estados y cambiar el comportamiento del script en consecuencia.

Funciones

Las funciones son bloques de código organizados, reutilizables y con un nombre asignado que se ejecutan para realizar una única tarea específica. Permiten modularizar el código, reciben argumentos de entrada (parámetros) y pueden devolver un resultado mediante la instrucción `return`.

Mark Lutz destaca que las funciones son la herramienta principal para evitar la duplicación de código (principio DRY - Don't Repeat Yourself). Las define como "unidades de empaquetamiento" que dividen un problema complejo en subtarear más pequeñas y administrables.

Listas

Las listas son estructuras de datos secuenciales, mutables (modificables) y ordenadas que permiten almacenar una colección de elementos. En Python, pueden contener objetos de diferentes tipos de datos y se indexan numéricamente comenzando desde el cero (0).

Charles Severance las describe desde una perspectiva pedagógica como un "contenedor lineal". Explica que las listas mantienen un orden posicional estricto de izquierda a derecha, ideal para almacenar colecciones donde el orden de los factores o la posición del elemento es un dato relevante.

Diccionarios

Los diccionarios son colecciones de datos mutables que implementan un arreglo asociativo o tabla de dispersión (hash table). A diferencia de las listas, no indexan por posición numérica, sino mediante claves únicas e inmutables, mapeando una relación directa de pares clave: valor.

John V. Guttag señala que los diccionarios representan una de las características más potentes de Python debido a su eficiencia de búsqueda (complejidad de tiempo constante o $O(1)$). Los define como "tablas de búsqueda optimizadas" donde el acceso al dato es casi instantáneo sin importar el volumen de elementos.

Ordenamientos (Algoritmo Bubble Sort / Método Burbuja)

Los ordenamientos son procesos algorítmicos diseñados para reorganizar los elementos de una lista en un orden secuencial específico (numérico o alfabético, ya sea ascendente o descendente). El método de la Burbuja compara sistemáticamente pares de elementos adyacentes y los intercambia si están en el orden incorrecto.

Thomas H. Cormen, un referente mundial en algoritmos, clasifica al Bubble Sort como un algoritmo de ordenamiento comparativo por pasadas. Explica que, aunque no es eficiente para grandes volúmenes de datos debido a su complejidad cuadrática ($O(n^2)$), su valor radica en su simplicidad pedagógica para entender la lógica de manipulación de punteros e índices.

Estadísticas Básicas en Programación

Concepto: Operaciones algorítmicas aplicadas sobre colecciones numéricas para extraer datos analíticos significativos, tales como valores extremos (Mínimos y Máximos), acumulaciones (Sumatorias), recuentos (Frecuencias) y promedios (Media Aritmética).

Wes McKinney (creador de la librería Pandas) aborda este concepto explicando que la computación aplicada a datos requiere transformar datos crudos en métricas de resumen. En la programación pura (sin librerías), esto sirve para entrenar la lógica de bucles acumuladores y estructuras condicionales de comparación.

Manejo de Excepciones (Try-Except)

El manejo de excepciones es un mecanismo que permite detectar, controlar y responder a errores durante la ejecución de un programa sin interrumpir abruptamente su funcionamiento. En Python, este control se realiza mediante las estructuras try, except, else y finally, que permiten separar la lógica principal del tratamiento de errores.

Las excepciones facilitan el desarrollo de aplicaciones más robustas, ya que permiten validar datos ingresados por el usuario, gestionar errores de conversión de tipos de datos, controlar accesos a archivos y manejar situaciones inesperadas de manera segura.

Mark Lutz (2013) sostiene que las excepciones constituyen el principal mecanismo de gestión de errores en Python. Según el autor, su utilización permite separar claramente el código destinado a resolver un problema del código encargado de manejar situaciones excepcionales, mejorando significativamente la legibilidad, mantenibilidad y confiabilidad del software.

Persistencia de Datos (Archivos CSV)

La persistencia de datos es la capacidad de un sistema informático para almacenar información de manera permanente, permitiendo que los datos continúen disponibles incluso después de finalizar la ejecución del programa.

Luciano Ramalho explica la interacción de Python con la persistencia de datos en disco. Describe el procesamiento de archivos estructurados (como Comma-Separated Values

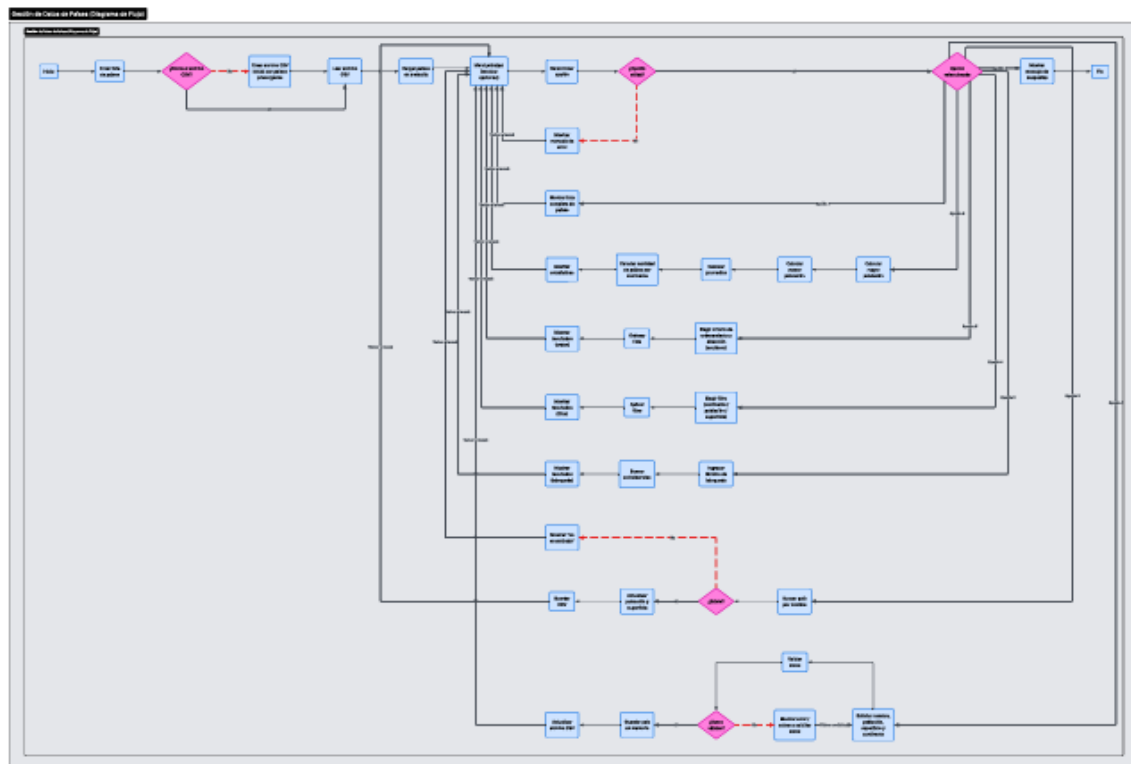
- CSV) línea por línea como un flujo continuo de cadenas de caracteres (streams) que el programa debe parsear (dividir usando el método `.split(',')`) para transformar y reconstruir objetos estructurados en la memoria volátil, tales como listas de diccionarios.

DECISIONES TÉCNICAS Y ARQUITECTURA

Para representar las entidades en la memoria del programa se optó por una combinación estructurada de colecciones: una Lista de Diccionarios. Cada país constituye un diccionario individual con claves definidas para sus atributos estables ('nombre', 'poblacion', 'superficie', 'continente'). Estos diccionarios son contenidos de forma secuencial dentro de una lista maestra. Esta decisión técnica equilibra la flexibilidad posicional de las listas con el acceso semántico inmediato que proveen las claves de los diccionarios.

Diagrama de Flujo Principal

A continuación se incorpora la representación gráfica de la arquitectura lógica del sistema, detallando el flujo de ejecución desde la verificación de persistencia en disco hasta el despliegue del menú y sus subprocesos correspondientes:



DIFICULTADES Y CONCLUSIONES

La realización de este Trabajo Práctico Integrador permitió consolidar los conocimientos teóricos adquiridos durante el cursado de Programación I, aplicándolos en el desarrollo de una solución funcional para la gestión de información de países. El proyecto integró conceptos fundamentales de programación estructurada, validación de datos, manipulación de estructuras dinámicas, persistencia de información y diseño modular, permitiendo transformar requerimientos funcionales en un sistema operativo y mantenible.

Desde el punto de vista del diseño, la utilización de listas de diccionarios facilitó la representación de entidades complejas mediante una estructura flexible y de fácil acceso. Cada país fue modelado como un conjunto de atributos relacionados, permitiendo realizar operaciones de búsqueda, filtrado, ordenamiento y actualización de manera eficiente. Asimismo, la implementación de funciones específicas para cada responsabilidad contribuyó a mejorar la legibilidad, reutilización y mantenimiento del código, siguiendo principios básicos de programación modular.

La incorporación de archivos CSV como mecanismo de persistencia permitió conservar la información entre ejecuciones del programa, acercando el desarrollo a escenarios reales donde los datos deben mantenerse disponibles más allá del ciclo de vida de una ejecución en memoria. Además, la validación sistemática de entradas mediante estructuras condicionales y manejo de excepciones fortaleció la robustez del sistema frente a errores de usuario.

Durante el desarrollo del proyecto se identificaron diversas dificultades propias del proceso de aprendizaje y de la implementación práctica de los conceptos estudiados. Uno de los principales desafíos fue diseñar correctamente la lógica necesaria para generar estadísticas a partir de los datos almacenados, relacionando adecuadamente la información de población, superficie y continente de cada país para obtener resultados coherentes y representativos.

Asimismo, resultó necesario analizar cómo estructurar y organizar la información ingresada por el usuario para que pudiera ser utilizada posteriormente en búsquedas, filtrados y procesos de ordenamiento. La implementación de criterios de agrupación por rangos de población y superficie también representó un desafío, ya que requirió definir condiciones lógicas precisas que permitieran clasificar correctamente los registros según los parámetros establecidos.

Otra dificultad estuvo relacionada con la persistencia de datos mediante archivos CSV. Durante las primeras etapas del desarrollo surgieron inconvenientes vinculados a las rutas de acceso a los archivos, provocando que el sistema no pudiera localizar o cargar correctamente la información almacenada. La resolución de este problema permitió

comprender mejor el funcionamiento de los directorios de trabajo y la gestión de archivos dentro de una aplicación.

Por último, una limitación del sistema radica en que su funcionamiento se desarrolla completamente a través de una interfaz de consola. Si bien este enfoque resulta adecuado para fines académicos y para comprender la lógica de programación, presenta restricciones en términos de experiencia de usuario en comparación con aplicaciones que disponen de interfaces gráficas más intuitivas y amigables.

En conclusión, el proyecto permitió aplicar de forma práctica los contenidos fundamentales de la asignatura, comprendiendo no solo el funcionamiento de cada estructura de programación utilizada, sino también la importancia del diseño, la organización y la validación de datos en la construcción de soluciones de software confiables y escalables.

BIBLIOGRAFÍA / WEBGRAFÍA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3a ed.). MIT Press.
- Downey, A. B. (2016). Think Python: How to Think Like a Computer Scientist (2a ed.). O'Reilly Media.
- Guttag, J. V. (2016). Introduction to Computation and Programming Using Python (2a ed.). MIT Press.
- Lutz, M. (2013). Learning Python (5a ed.). O'Reilly Media.
- McKinney, W. (2017). Python for Data Analysis (2a ed.). O'Reilly Media.
- Ramalho, L. (2015). Fluent Python: Clear, Concise, and Effective Programming. O'Reilly Media.
- Severance, C. R. (2020). Python para todos: Explorando la información con Python 3. Edición Independiente.

ENLACES DEL PROYECTO

- Enlace al Repositorio de GitHub: <https://github.com/sannnew/TPI-programacion-paises/upload/main>
- Enlace al Video de Demostración: [https://youtu.be/UMwC5_cdIC8]